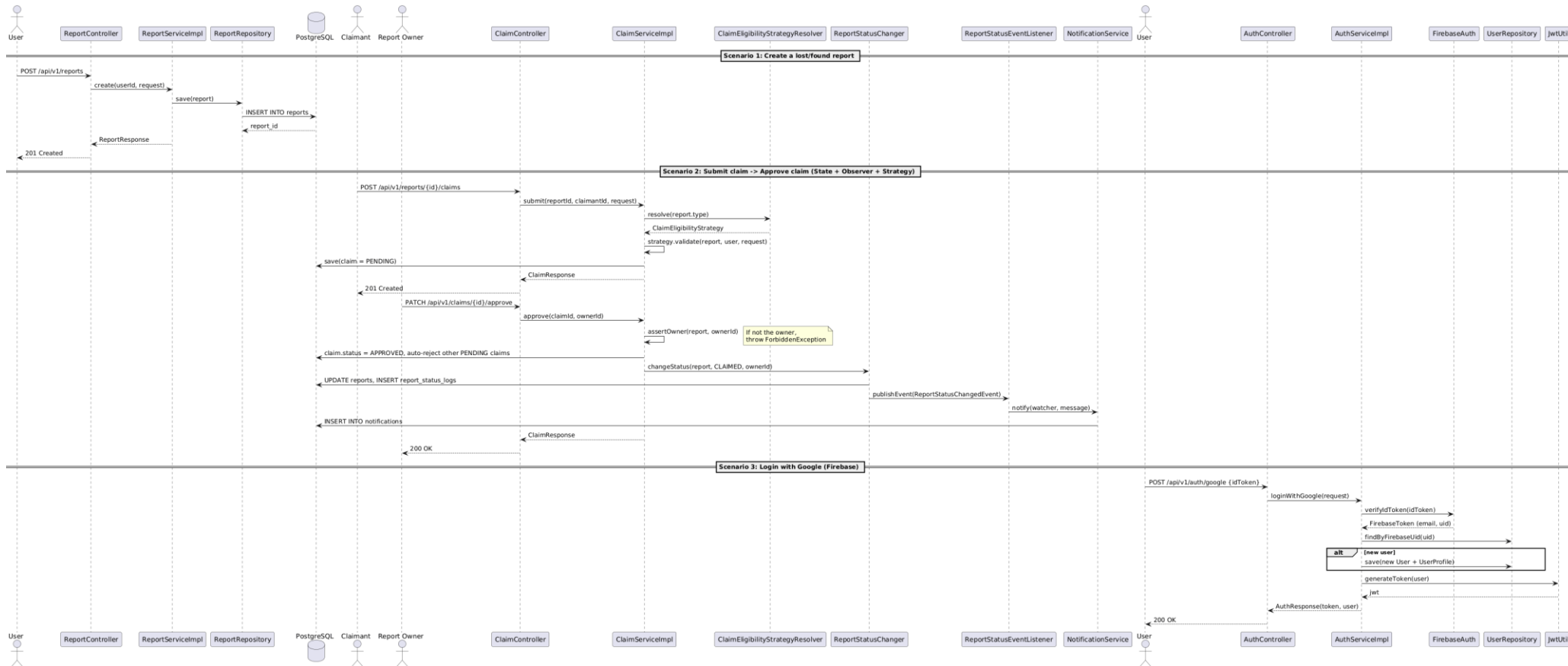


Sequence Diagram



ไดอะแกรมนี้แสดงลำดับขั้นตอนการทำงาน (Interaction) ระหว่าง Component ต่างๆ ตั้งแต่ฝั่ง Client (User) ผ่าน Controller, Service, Repository ไปจนถึงฐานข้อมูล (PostgreSQL) และระบบภายนอก (Firebase) โดยเรียงลำดับตามเวลาจากบนลงล่าง

องค์ประกอบหลัก (Lifelines) ที่ปรากฏ

- Actors: User, Claimant, Report Owner
- Controllers: ReportController, ClaimController, AuthController
- Services: ReportServiceImpl, ClaimServiceImpl, AuthServiceImpl, NotificationService
- Resolvers/Handlers: ClaimEligibilityStrategyResolver, ReportStatusChanger, ReportStatusEventListener

- Repositories: ReportRepository, UserRepository
- External/Database: PostgreSQL, FirebaseAuth, JwtUtil

Scenario 1: Create a lost/found report (การสร้าง Report ของหาย/ของเจอ)

แสดงขั้นตอนการที่ผู้ใช้สร้าง Report ใหม่เข้าสู่ระบบ

ลำดับการทำงาน

1. User ส่งคำขอ POST /api/v1/reports ไปยัง ReportController
2. ReportController เรียกเมธอด create(userId, request) ไปยัง ReportServiceImpl
3. ReportServiceImpl เรียกเมธอด save(report) ไปยัง ReportRepository
4. ReportRepository ทำคำสั่ง INSERT INTO reports ลงใน PostgreSQL
5. PostgreSQL ส่ง report_id กลับมาให้ ReportRepository
6. ReportRepository ส่งข้อมูล Report ที่บันทึกแล้วกลับไปให้ ReportServiceImpl
7. ReportServiceImpl ส่ง ReportResponse กลับไปให้ ReportController
8. ReportController ส่งสถานะ 201 Created กลับไปให้ User

Scenario 2: Submit claim -> Approve claim (State + Observer + Strategy)

(การส่ง Claim และการอนุมัติ Claim โดยใช้ Design Patterns)

แสดงขั้นตอนการที่ผู้ใช้ส่ง Claim เพื่อขอรับของคืน และเจ้าของ Report เป็นผู้อนุมัติ โดยมีการใช้ Design Patterns 3 ตัวร่วมกัน คือ State, Observer, และ Strategy

ลำดับการทำงาน

ส่วนที่ 1: Submit claim (การส่ง Claim)

1. Claimant ส่งคำขอ POST /api/v1/reports/{id}/claims ไปยัง ClaimController
2. ClaimController เรียกเมธอด submit(reportId, claimantId, request) ไปยัง ClaimServiceImpl
3. ClaimServiceImpl เรียกเมธอด resolve(report.type) ไปยัง ClaimEligibilityStrategyResolver เพื่อเลือก Strategy ที่เหมาะสม (Strategy Pattern)
4. ClaimEligibilityStrategyResolver ส่ง ClaimEligibilityStrategy กลับมา

5. ClaimServiceImpl เรียกเมธอด strategy.validate(report, user, request) เพื่อตรวจสอบกฎการ Claim
6. ClaimServiceImpl เรียกเมธอด save(claim = PENDING) ไปยัง ReportRepository (หรือ ClaimRepository ที่ไม่ได้แสดงในรูป)
7. ReportRepository ทำคำสั่ง INSERT INTO claims ลงใน PostgreSQL
8. ClaimServiceImpl ส่ง ClaimResponse กลับไปให้ ClaimController
9. ClaimController ส่งสถานะ 201 Created กลับไปให้ Claimant

ส่วนที่ 2: Approve claim (การอนุมัติ Claim)

1. Report Owner ส่งคำขอ PATCH /api/v1/claims/{id}/approve ไปยัง ClaimController
2. ClaimController เรียกเมธอด approve(claimId, ownerId) ไปยัง ClaimServiceImpl
3. ClaimServiceImpl เรียกเมธอด assertOwner(report, ownerId) เพื่อตรวจสอบสิทธิ์
 - หมายเหตุสำคัญ: หากไม่ใช่เจ้าของ จะ throw ForbiddenException (โยนข้อผิดพลาด)
4. ClaimServiceImpl เปลี่ยนสถานะ claim.status = APPROVED และทำการ auto-reject other PENDING claims (ปฏิเสธ Claim อื่นที่ค้างอยู่บน Report เดียวกันโดยอัตโนมัติ)
5. ClaimServiceImpl เรียกเมธอด changeStatus(report, CLAIMED, ownerId) ไปยัง ReportStatusChanger (State Pattern)
6. ReportStatusChanger ทำคำสั่ง UPDATE reports, INSERT report_status_logs ลงใน PostgreSQL
7. ReportStatusChanger เรียกเมธอด publishEvent(ReportStatusChangedEvent) ไปยัง ReportStatusEventListener (Observer Pattern)
8. ReportStatusEventListener เรียกเมธอด notify(watcher, message) ไปยัง NotificationService
9. NotificationService ทำคำสั่ง INSERT INTO notifications ลงใน PostgreSQL
10. ClaimServiceImpl ส่ง ClaimResponse กลับไปให้ ClaimController
11. ClaimController ส่งสถานะ 200 OK กลับไปให้ Report Owner

Scenario 3: Login with Google (Firebase)

(การเข้าสู่ระบบด้วย Google ผ่าน Firebase)

แสดงขั้นตอนการที่ผู้ใช้เข้าสู่ระบบโดยใช้บัญชี Google ผ่าน Firebase Authentication และการจัดการผู้ใช้ใหม่ในฐานข้อมูล

ลำดับการทำงาน

1. User ส่งคำขอ POST /api/v1/auth/google (idToken) ไปยัง AuthController
2. AuthController เรียกเมธอด loginWithGoogle(request) ไปยัง AuthServiceImpl
3. AuthServiceImpl เรียกเมธอด verifyIdToken(idToken) ไปยัง FirebaseAuth (External Service)
4. FirebaseAuth ส่งข้อมูล FirebaseToken (email, uid) กลับมา
5. AuthServiceImpl เรียกเมธอด findByFirebaseUid(uid) ไปยัง UserRepository
6. UserRepository ค้นหาข้อมูลใน PostgreSQL
7. เงื่อนไข (alt): หากเป็นผู้ใช้ใหม่ ([new user])
 - AuthServiceImpl เรียกเมธอด save(new User + UserProfile) ไปยัง UserRepository เพื่อบันทึกผู้ใช้ใหม่ลงฐานข้อมูล
 - UserRepository ทำคำสั่ง INSERT INTO users, user_profiles ลงใน PostgreSQL
8. AuthServiceImpl เรียกเมธอด generateToken(user) ไปยัง JwtUtil เพื่อสร้าง Token สำหรับการใช้งานในระบบ
9. JwtUtil ส่ง jwt กลับมา
10. AuthServiceImpl ส่ง AuthResponse(token, user) กลับไปให้ AuthController
11. AuthController ส่งสถานะ 200 OK กลับไปให้ User